

Lab 9 Study and Implementation of Abstract Class and Interface in Java

Objectives

- To identify limitations of normal class design
 - To implement abstract classes for partial abstraction
 - To implement interfaces for complete abstraction
 - To compare design, flexibility, and correctness
-

Theory

In object-oriented programming:

- A normal class may lead to generic or incorrect behavior when subclasses require different logic.
- An abstract class provides partial abstraction, enforcing implementation while allowing shared code.
- An interface provides complete abstraction, defining only rules that must be implemented.

Thus:

- Abstract class → “*What + some how*”
 - Interface → “*Only what*”
-

Exercise 1: Payment Processing System

Part A: Normal Class

```
class Payment {  
    double amount;  
  
    void pay() {  
        System.out.println("Processing payment of " + amount);  
    }  
}
```

Part B: Abstract Class

```
abstract class Payment {
    double amount;

    Payment(double amount) {
        this.amount = amount;
    }

    abstract void pay();
}

class UPI extends Payment {
    String upiId;

    UPI(double amount, String upiId) {
        super(amount);
        this.upiId = upiId;
    }

    void pay() {
        System.out.println("UPI Payment: " + amount + " using " + upiId);
    }
}
```

Part C: Interface

```
interface Payment {
    void pay();
}

class UPI implements Payment {
    double amount;
    String upiId;

    UPI(double amount, String upiId) {
        this.amount = amount;
        this.upiId = upiId;
    }

    public void pay() {
        System.out.println("UPI Payment: " + amount + " using " + upiId);
    }
}
```

Observation

- Normal class → generic behavior
 - Abstract class → controlled and structured
 - Interface → rule-based implementation
-

Exercise 2: Employee Salary System

Abstract Class

```
abstract class Employee {
    double base;

    Employee(double base) {
        this.base = base;
    }

    abstract void calculateSalary();
}

class FullTime extends Employee {
    FullTime(double base) {
        super(base);
    }

    void calculateSalary() {
        System.out.println("Salary: " + (base + 5000));
    }
}
```

Interface

```
interface Employee {
    void calculateSalary();
}

class Intern implements Employee {
    double stipend;

    Intern(double stipend) {
        this.stipend = stipend;
    }

    public void calculateSalary() {
        System.out.println("Stipend: " + stipend);
    }
}
```

```
}  
}
```

Exercise 3: Notification System

Abstract Class

```
abstract class Notification {  
    abstract void send();  
}  
  
class Email extends Notification {  
    void send() {  
        System.out.println("Sending Email");  
    }  
}
```

Interface

```
interface Notification {  
    void send();  
}  
  
class SMS implements Notification {  
    public void send() {  
        System.out.println("Sending SMS");  
    }  
}
```

Exercise 4: Vehicle Fare System

Abstract Class

```
abstract class Vehicle {  
    double distance;  
  
    Vehicle(double distance) {  
        this.distance = distance;  
    }  
  
    abstract void fare();  
}  
  
class Car extends Vehicle {  
    Car(double d) {  
        super(d);  
    }  
}
```

```
}  
  
void fare() {  
    System.out.println("Car Fare: " + distance * 15);  
}  
}
```

Interface

```
interface Vehicle {  
    void fare();  
}
```

```
class Bike implements Vehicle {  
    double distance;  
  
    Bike(double d) {  
        this.distance = d;  
    }  
  
    public void fare() {  
        System.out.println("Bike Fare: " + distance * 8);  
    }  
}
```

1. What problem is observed in normal class design?
 2. How does an abstract class improve correctness?
 3. How does an interface differ from an abstract class?
 4. Which approach enforces implementation more strictly?
 5. Which design is more flexible for future extension?
-

Conceptual Questions

1. When would you prefer an abstract class over an interface?
2. When is an interface more suitable?
3. Can a class implement multiple interfaces? Explain.